FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO

DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE

Plataforma IoT de Monitoramento para Criação de Bubalinos

Arthur Fukunaga
Leo De Lima
Matheus Ferrari
Vinícius Souza

Registro – SP
2026

Índice

1 CANVAS: Modelo de Negócio e Estratégia **3**

2 SWOT: Análise Estratégica **3**
2.1 Recomendações Estratégicas 4

3 UX/UI: Design de Interface e Experiência do Usuário **4**
3.1 Guia de Estilos 4

4 Website: Estrutura e Especificações Técnicas Web **5**
4.1 Mapa do Site da Equipe 5
4.2 Conteúdo das Páginas 6
4.3 Especificações Técnicas do Site da Equipe 6

5 Aplicação: Estrutura e Especificações **6**
5.1 Mapa do Sistema Web 6
5.2 Conteúdo das Páginas 7
5.3 Especificações Técnicas do Sistema Web 8

6 Diagramas UML: Modelagem Visual do Sistema **8**
6.1 Diagrama de Casos de Uso 8
6.2 Diagrama de Classes 9
6.3 Diagrama de Objetos 10
6.4 Outros Diagramas UML Importantes 11

7 Diagrama de Banco de Dados **11**
7.1 Diagrama Conceitual 11
7.2 Diagrama Lógico 12

8 DevOps: Infraestrutura, Automação e Entrega Contínua **13**
8.1 Pipeline CI/CD 13
8.2 Exemplo de Configuração CI/CD 14
8.3 Infraestrutura como Código 16
8.4 Infraestrutura do Sistema 16
8.5 Arquitetura de Rede 16
8.6 Diagrama de Infraestrutura 17
8.7 Arquitetura de Deploy 19
8.8 Métricas e SLA 20

9 Conclusão **21**

Referências **22**

Lista de Artefatos e suas Finalidades

- **CANVAS:** Modelo de negócio e estratégia
- **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
- **UX/UI:** Design de interface e experiência do usuário
- **Website:** Estrutura e especificações técnicas web
- **Aplicação:** Funcionalidades, arquitetura e tecnologias utilizadas no sistema
- **Diagramas UML:** Modelagem visual do sistema
- **Diagramas de Banco de Dados:** Modelagem visual do sistema
- **DevOps:** Infraestrutura, automação e práticas de entrega contínua

---

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas (**??**) é uma ferramenta visual estratégica[1] que permite descrever, analisar e projetar modelos de negócio de forma clara e objetiva. O canvas é dividido em nove blocos fundamentais que cobrem as principais áreas de um negócio.

Tabela 1: Business Model Canvas – Visão Geral



Figura 1: Business Model Canvas do Projeto

2 SWOT: Análise Estratégica

A Análise SWOT (Strengths, Weaknesses, Opportunities, Threats) ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) (**??**) é uma ferramenta fundamental para planejamento estratégico[2] que permite identificar fatores internos e externos que impactam o projeto. Analisa forças, fraquezas, oportunidades e ameaças do projeto.

A matriz SWOT pode ser apresentada em formato de tabela conforme o exemplo abaixo:

[1] O Canvas foi desenvolvido por Alexander Osterwalder e é amplamente utilizado em startups e empresas estabelecidas para modelagem de negócios.

[2] A análise SWOT foi criada na década de 1960 por Albert Humphrey no Stanford Research Institute.

Tabela 2: Análise SWOT do Projeto



Figura 2: Análise SWOT do Projeto

2.1 Recomendações Estratégicas

Com base na análise SWOT, as principais estratégias devem focar em:

- **Forças + Oportunidades:** Investir no marketing digital para aproveitar o crescimento do mercado
- **Forças + Ameaças:** Reforçar a segurança para se diferenciar da concorrência
- **Fraquezas + Oportunidades:** Buscar parcerias para ampliar a presença no mercado
- **Fraquezas + Ameaças:** Criar planos de contingência e melhorar a documentação

3 UX/UI: Design de Interface e Experiência do Usuário

UX (User Experience) e UI (User Interface) (**??**) são disciplinas complementares focadas em criar produtos digitais que sejam funcionais, acessíveis e agradáveis de usar.[3] O UX concentra-se na experiência geral do usuário, enquanto o UI cuida dos elementos visuais e interativos.

3.1 Guia de Estilos

É importante documentar o design system utilizado:

[3]Ferramentas populares para design UX/UI incluem Figma, Adobe XD, Sketch e InVision.

Tabela 3: Paleta de Cores do Site da Equipe

Elemento	Cor	Código HEX	Uso
Primária		#E38B18	Destaques e títulos
Secundária		#12C400	Botões e ações
Fundo Principal		#ECECEC	Fundo das páginas
Fundo Secundário		#3F3F3F	Navbar e cards
Texto Principal		#111111	Textos principais
Texto Secundário		#FFFFFF	Textos em destaque
Sucesso		#22C55E	Confirmações
Alerta		#F59E0B	Avisos
Erro		#EF4444	Mensagens de erro

Tabela 4: Paleta de Cores da Aplicação

Elemento	Cor	Código HEX	Uso
Primária		#07DB00	Destaques e botões
Primária escura		#027300	Degradê do logotipo
Secundária		#36454F	Superfícies neutras e hover
Fundo Principal		#1B262C	Fundo do dashboard
Fundo Secundário		#2D3B43	Painéis internos
Cards e janelas		#243138	Cards, tabelas e modais
Menu lateral		#151E23	Menu e painel do login
Bordas		#38454D	Contornos de cards e campos
Texto Principal		#F3F6F5	Títulos e textos
Texto Secundário		#A4B1B7	Informações auxiliares
Sucesso		#05D900	Status positivos
Alerta		#FFB703	Avisos
Erro		#FF4D5A	Mensagens de erro e estresse
Oliva		#90A955	Status sem classificação
Cinza		#6B7A84	Status sem leitura e inativo

4 Website: Estrutura e Especificações Técnicas Web

Esta seção documenta o website institucional/portfólio da equipe que desenvolveu a aplicação. O site da equipe serve para apresentar os membros, projetos realizados, competências e informações de contato.

4.1 Mapa do Site da Equipe

O mapa do site (sitemap) mostra a hierarquia e organização das páginas do website da equipe:

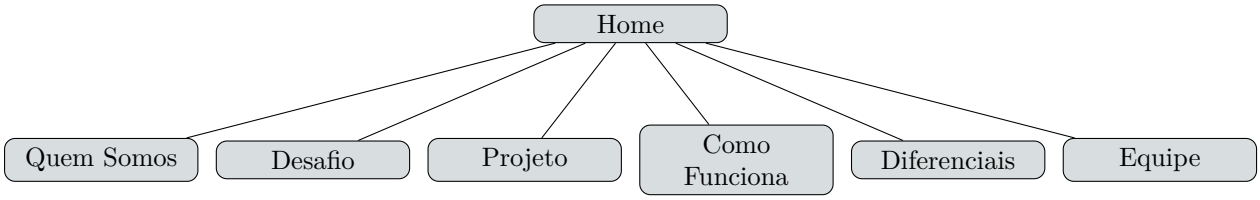


Figura 3: Estrutura de navegação do website da equipe

4.2 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Home:** landing page do website
- **Quem Somos:** história da equipe, missão, visão e valores
- **Desafio:** apresentação do problema que o projeto busca resolver
- **Projeto:** apresentação do produto da equipe, o Bubatag, sistema de monitoramento de estresse e localização de bubalinos
- **Como Funciona:** explicação do funcionamento da solução
- **Diferenciais:** vantagens e pontos fortes da solução
- **Equipe:** apresentação dos membros da equipe

4.3 Especificações Técnicas do Site da Equipe

Tabela 5: Stack Tecnológica do Website da Equipe

Camada	Tecnologia	Descrição
Frontend	React 19.2.4	Biblioteca para interfaces web
Framework	Next.js 16.2.6	Framework React para aplicações modernas
Linguagem	TypeScript 5	Tipagem estática para JavaScript
Estilização	Tailwind CSS v4	Framework CSS responsivo
Pós-processamento CSS	PostCSS	Processamento de estilos CSS
Animações	AOS 2.3.4	Animações durante rolagem
Qualidade de Código	ESLint 9	Padronização e análise de código
Compilação	React Compiler	Otimização de performance React

5 Aplicação: Estrutura e Especificações

Esta seção documenta a estrutura do sistema web desenvolvido para monitoramento inteligente de bubalinos utilizando tecnologias IoT. O website permite o gerenciamento dos animais, coleiras inteligentes, sensores e monitoramento em tempo real dos dados coletados no ambiente rural.

5.1 Mapa do Sistema Web

O mapa do site (sitemap) apresenta a hierarquia e organização das páginas da aplicação web.

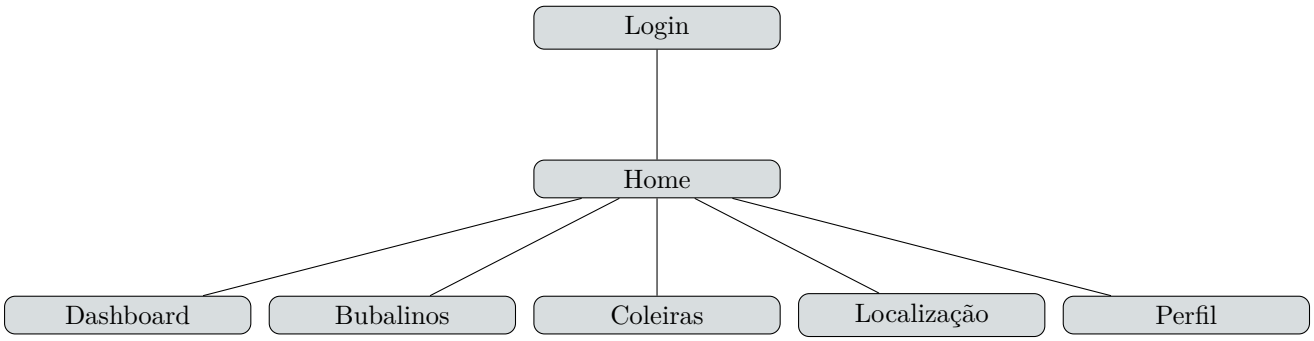


Figura 4: Estrutura de navegação do sistema web

5.2 Conteúdo das Páginas

Principais páginas do sistema web:

- **Login:** Responsável pela autenticação e acesso dos usuários ao sistema.
- **Home:** Página inicial do sistema, responsável por centralizar o acesso às principais funcionalidades da aplicação.
- **Dashboard:** Exibe métricas em tempo real relacionadas à temperatura corporal, batimentos cardíacos e demais informações coletadas pelas coleiras inteligentes.
- **Bubalinos:** Página destinada ao cadastro, visualização e gerenciamento dos animais monitorados.
- **Coleiras:** Responsável pelo gerenciamento das coleiras inteligentes vinculadas aos animais.
- **Localização:** Permite visualizar a localização em tempo real dos bubalinos monitorados.
- **Perfil:** Página destinada ao gerenciamento das informações do usuário e configurações da conta.

5.3 Especificações Técnicas do Sistema Web

Tabela 6: Stack Tecnológica do Sistema Web

Camada	Tecnologia	Descrição
Frontend	React 18.2.0	Biblioteca JavaScript para construção de interfaces reativas.
Build Tool	Vite 6.1.0	Ferramenta de build e servidor de desenvolvimento para aplicações web.
Linguagem	JavaScript (ES Modules)	Linguagem utilizada no desenvolvimento da aplicação.
Roteamento	React Router DOM 6.30.3	Gerenciamento de rotas em aplicações React.
Gerenciamento de Estado/ Cache	TanStack React Query 5.100.10	Gerenciamento de cache e requisições assíncronas.
Requisições HTTP	Axios 1.16.1	Cliente HTTP utilizado para comunicação com APIs.
Estilização	Tailwind CSS 3.4.17	Framework CSS utilitário para estilização responsiva.
Componentes UI	Radix UI	Biblioteca de componentes acessíveis e reutilizáveis.
Ícones	Lucide React 0.473.0	Biblioteca de ícones para aplicações React.
Mapas	Leaflet 1.9.4 + React Leaflet 4.2.1	Bibliotecas para renderização e interação com mapas.
Notificações	Sonner 1.5.0	Biblioteca para exibição de notificações toast.
Qualidade de Código	ESLint 10.3.0	Ferramenta para padronização e análise de código JavaScript.
Pós-processamento CSS	PostCSS + Autoprefixer	Ferramentas para processamento e compatibilidade de CSS.
Containerização	Docker + Docker Compose	Ferramentas para construção e execução de containers.
CI/CD	GitHub Actions	Automação da validação do Docker Compose e construção da imagem da aplicação.

6 Diagramas UML: Modelagem Visual do Sistema

A Unified Modeling Language (UML) (**??**) é uma linguagem de modelagem visual usada para especificar, visualizar, construir e documentar artefatos de sistemas de software.[4]

6.1 Diagrama de Casos de Uso

O diagrama de casos de uso representa as funcionalidades do sistema do ponto de vista do usuário:

[4] A UML foi padronizada pela Object Management Group (OMG) em 1997.

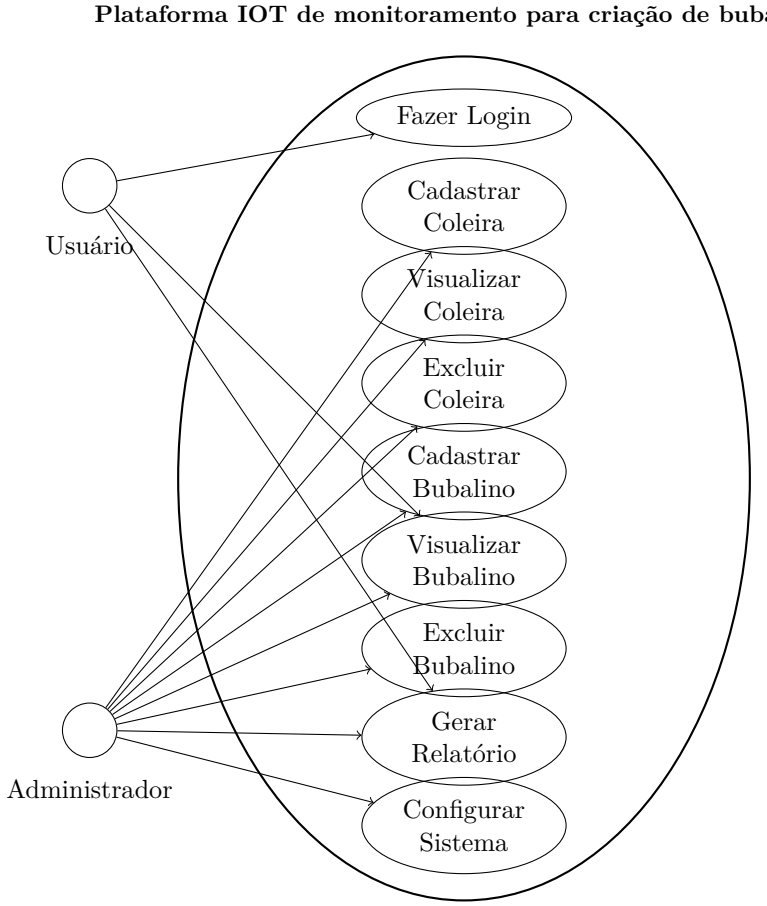


Figura 5: Diagrama de Casos de Uso

6.2 Diagrama de Classes

O diagrama de classes representa a estrutura estática do sistema, demonstrando as classes, atributos, métodos e os relacionamentos existentes entre os componentes da aplicação. Esse tipo de diagrama é amplamente utilizado para modelagem orientada a objetos, auxiliando na organização e desenvolvimento do software.

Por meio do diagrama de classes é possível visualizar a arquitetura lógica do sistema, facilitando a compreensão da interação entre entidades e regras de negócio.

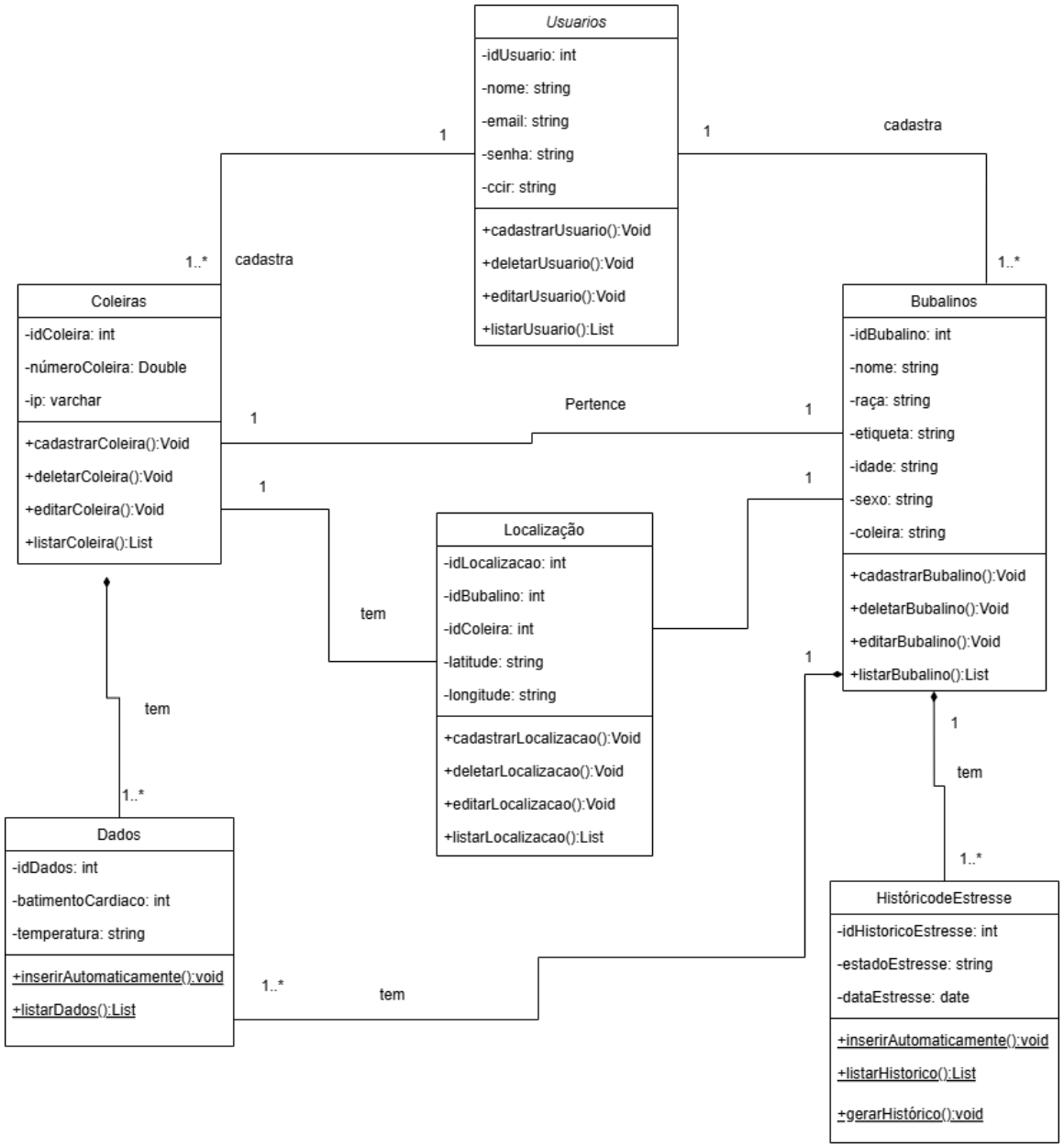


Figura 6: Diagrama de Classes do Sistema

6.3 Diagrama de Objetos

O diagrama de objetos representa instâncias específicas das classes em determinado momento da execução do sistema. Diferentemente do diagrama de classes, que apresenta uma visão genérica da estrutura do software, o diagrama de objetos demonstra exemplos concretos de objetos e seus relacionamentos.

Esse tipo de diagrama é utilizado para facilitar a visualização do comportamento das entidades e da troca de informações entre os componentes da aplicação.

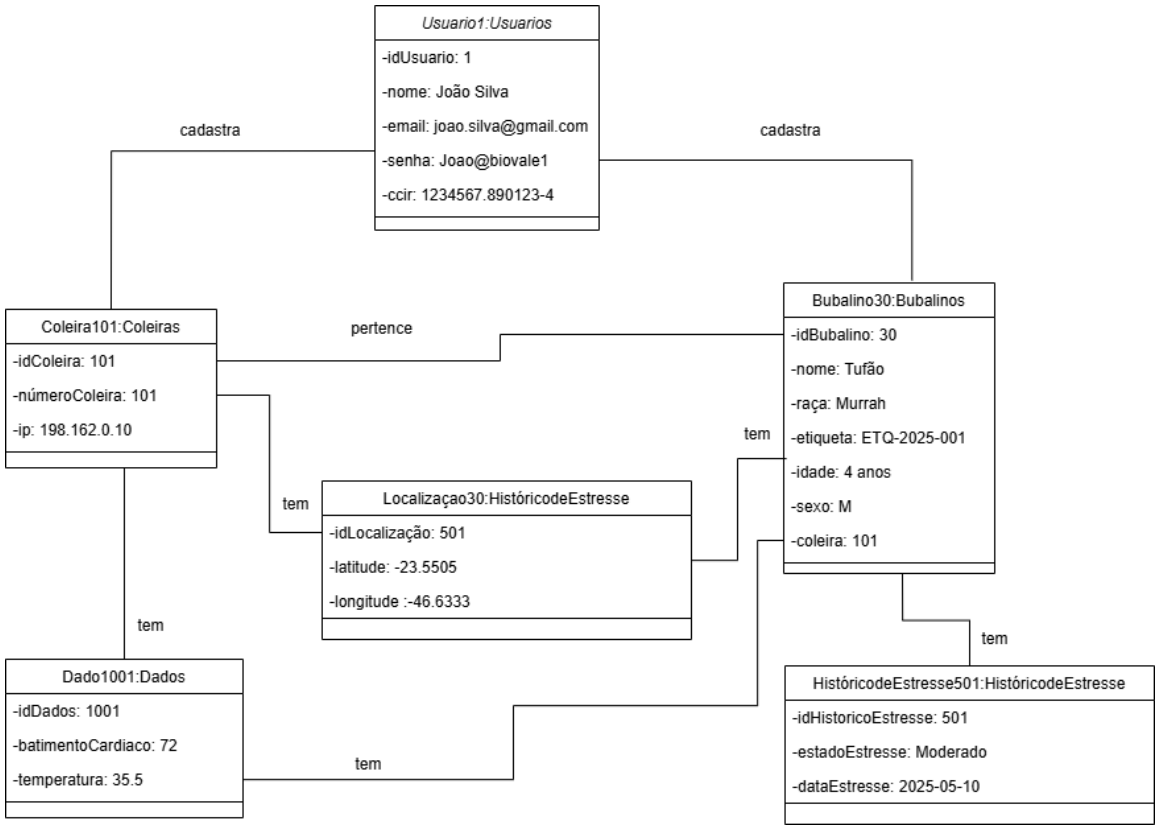


Figura 7: Diagrama de Objetos do Sistema

6.4 Outros Diagramas UML Importantes

- **Diagrama de Sequência:** Mostra a interação entre objetos ao longo do tempo
- **Diagrama de Atividades:** Representa o fluxo de trabalho ou processo
- **Diagrama de Componentes:** Mostra a organização física dos componentes
- **Diagrama de Implantação:** Representa a arquitetura física do sistema
- **Diagrama de Estados:** Mostra os estados de um objeto e suas transições

7 Diagrama de Banco de Dados

O Diagrama de Banco de Dados é uma representação visual da estrutura das informações armazenadas em um sistema (**??**). Esse tipo de diagrama permite modelar entidades, atributos e relacionamentos existentes no banco de dados, auxiliando no desenvolvimento, manutenção e organização das informações.

Além disso, os diagramas de banco de dados facilitam a compreensão da arquitetura de dados, contribuindo para garantir integridade, consistência e eficiência no armazenamento das informações. Esses diagramas podem ser apresentados utilizando modelos Entidade-Relacionamento (ER) ou modelos relacionais, conforme descrito nas subseções abaixo.

7.1 Diagrama Conceitual

O diagrama conceitual representa uma visão de alto nível da estrutura do banco de dados (**??**). Nesse modelo são identificadas as entidades principais do sistema, seus atributos e os relacionamentos existentes entre elas, sem considerar detalhes específicos de implementação.

O principal objetivo do modelo conceitual é facilitar a compreensão das regras de negócio e da organização das informações, servindo como base para as etapas posteriores de modelagem do banco de dados.

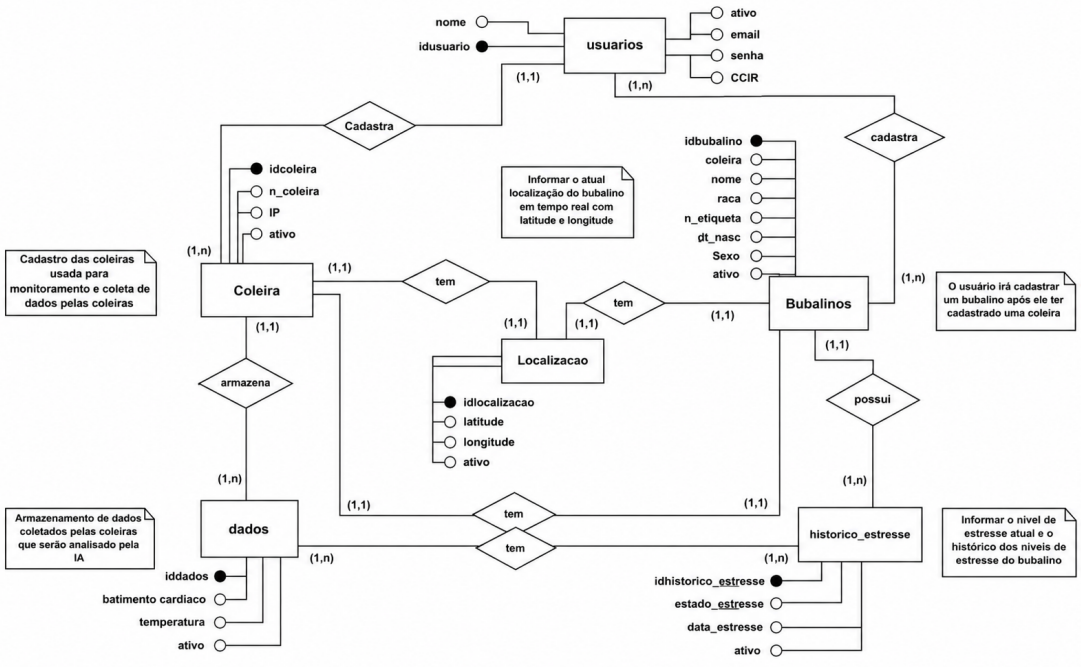


Figura 8: Diagrama Conceitual

7.2 Diagrama Lógico

O diagrama lógico detalha a estrutura do banco de dados considerando tabelas, atributos, chaves primárias, chaves estrangeiras e relacionamentos entre as entidades (**??**). Diferentemente do modelo conceitual, o modelo lógico apresenta maior proximidade com a implementação do banco de dados.

Esse modelo é utilizado para organizar os dados de maneira estruturada, garantindo consistência e integridade das informações armazenadas.

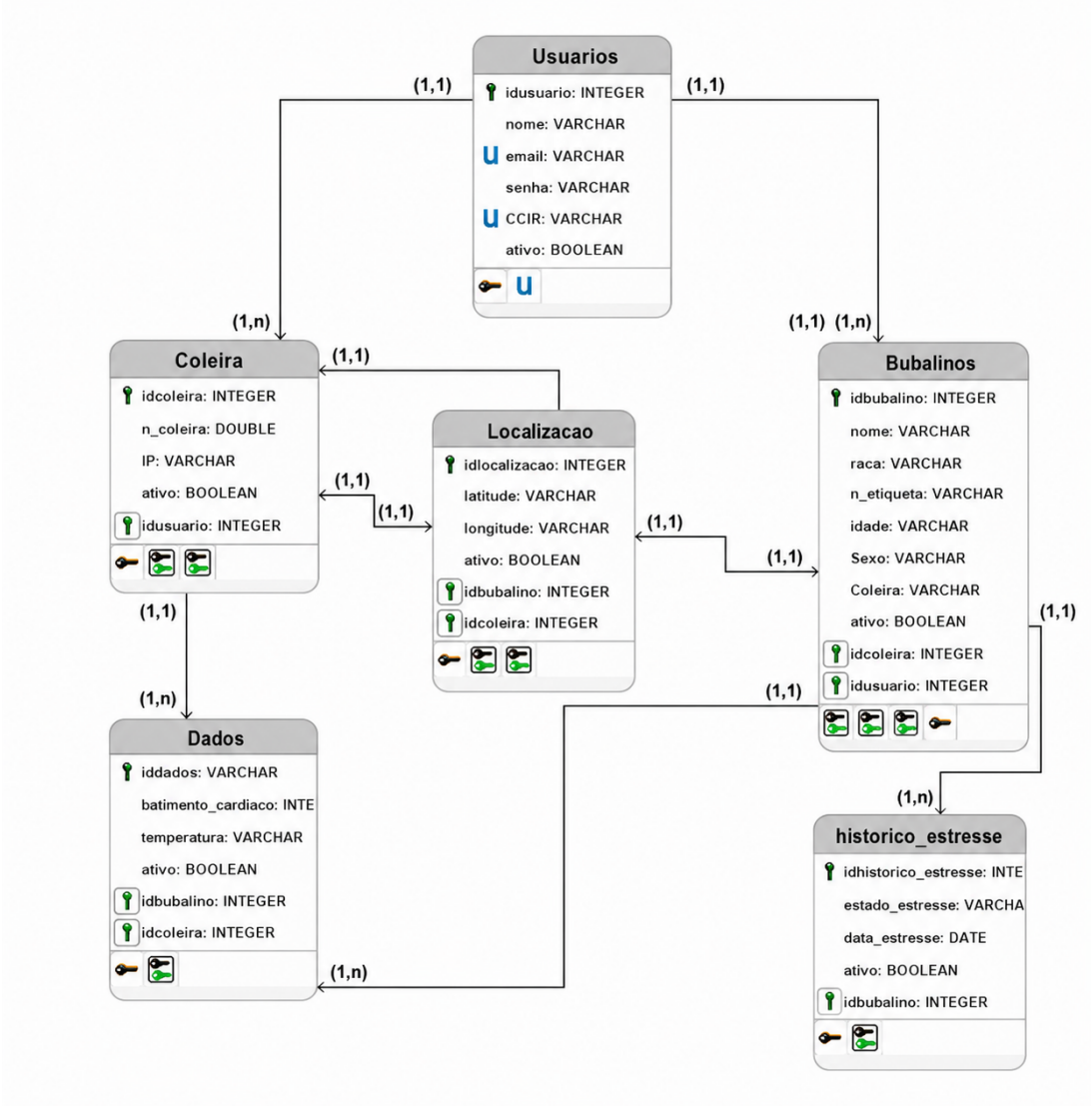


Figura 9: Diagrama Lógico

8 DevOps: Infraestrutura, Automação e Entrega Contínua

DevOps (**??**) é uma cultura e conjunto de práticas que integra desenvolvimento de software (Dev) e operações de TI (Ops), focando em automação, colaboração e entrega contínua.

8.1 Pipeline CI/CD

O pipeline de integração e entrega contínua é fundamental para automatizar os processos de validação e construção das aplicações:

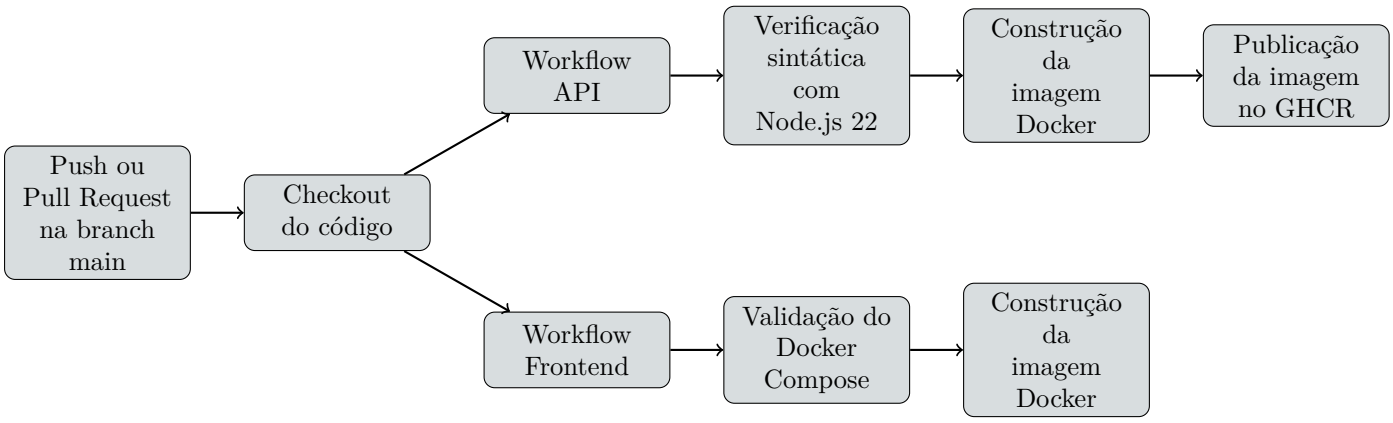


Figura 10: Fluxos de CI/CD dos repositórios do Bubatag

8.2 Exemplo de Configuração CI/CD

Os processos de integração contínua do Bubatag são automatizados por meio do GitHub Actions, utilizando arquivos YAML armazenados no diretório `.github/workflows` de cada repositório.

No frontend, o workflow é acionado por eventos de *push* e *pull request* direcionados à branch `main`. Suas etapas incluem a obtenção do código-fonte, a configuração do Docker Buildx, a validação do arquivo Docker Compose e a construção da imagem Docker da aplicação.

A seguir, apresenta-se a configuração utilizada no repositório do frontend:

```
name: CI/CD - Bubatag Frontend

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Baixar código
        uses: actions/checkout@v4

      - name: Configurar Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Validar Docker Compose
        run: docker compose -f ./sistema-front/docker-compose.yml config

      - name: Construir imagem do frontend
        run: docker compose -f ./sistema-front/docker-compose.yml build
```

Na API, o workflow também é acionado por eventos de *push* e *pull request* direcionados à branch `main`. O processo realiza a configuração do ambiente Node.js 22, a instalação das

dependências, a verificação sintática do código, a construção da imagem Docker e sua publicação no GitHub Container Registry (GHCR).

A configuração utilizada no repositório da API é apresentada a seguir:

```
name: CI/CD - Bubatag API

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read
  packages: write

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Baixar código
        uses: actions/checkout@v4

      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependências
        run: npm ci

      - name: Validar sintaxe
        run: node --check src/server.js

      - name: Construir imagem Docker
        run: docker build -t bubatag-api .

      - name: Login no GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: \${{ github.actor }}
          password: \${{ secrets.GITHUB_TOKEN }}

      - name: Publicar imagem Docker
        run: |
          docker tag bubatag-api ghcr.io/\${{ github.repository_owner }}/bubatag-api:latest
```

```
docker push ghcr.io/\${{ github.repository_owner }}/bubatag-api:latest
```

Dessa forma, os dois repositórios possuem processos automatizados de integração contínua. Enquanto o frontend realiza a validação da configuração do Docker Compose e a construção de sua imagem, a API também automatiza a instalação de dependências, a verificação sintática e a publicação da imagem Docker no GHCR.

8.3 Infraestrutura como Código

A infraestrutura do projeto Bubatag utiliza ferramentas de controle de versão, automação de processos e containerização, que auxiliam no gerenciamento e na construção das aplicações. A Tabela 7 apresenta as principais ferramentas utilizadas.

Tabela 7: Ferramentas DevOps Utilizadas

Categoria	**Ferramenta**	**Finalidade**
Controle de Versão	Git + GitHub	Versionamento e gerenciamento do código-fonte
CI/CD	GitHub Actions	Automação das etapas de validação e construção das aplicações
Containerização	Docker	Construção e empacotamento das aplicações em containers
Orquestração	Docker Compose	Configuração e execução de serviços containerizados
Registro de Imagens	GitHub Container Registry (GHCR)	Armazenamento e publicação da imagem Docker da API

8.4 Infraestrutura do Sistema

Esta seção apresenta a infraestrutura do sistema Bubatag, considerando os componentes de hardware, comunicação, armazenamento e aplicações utilizados no desenvolvimento do protótipo. A arquitetura permite a coleta de dados fisiológicos dos animais, seu envio para uma API e a disponibilização das informações nas interfaces web e mobile.

8.5 Arquitetura de Rede

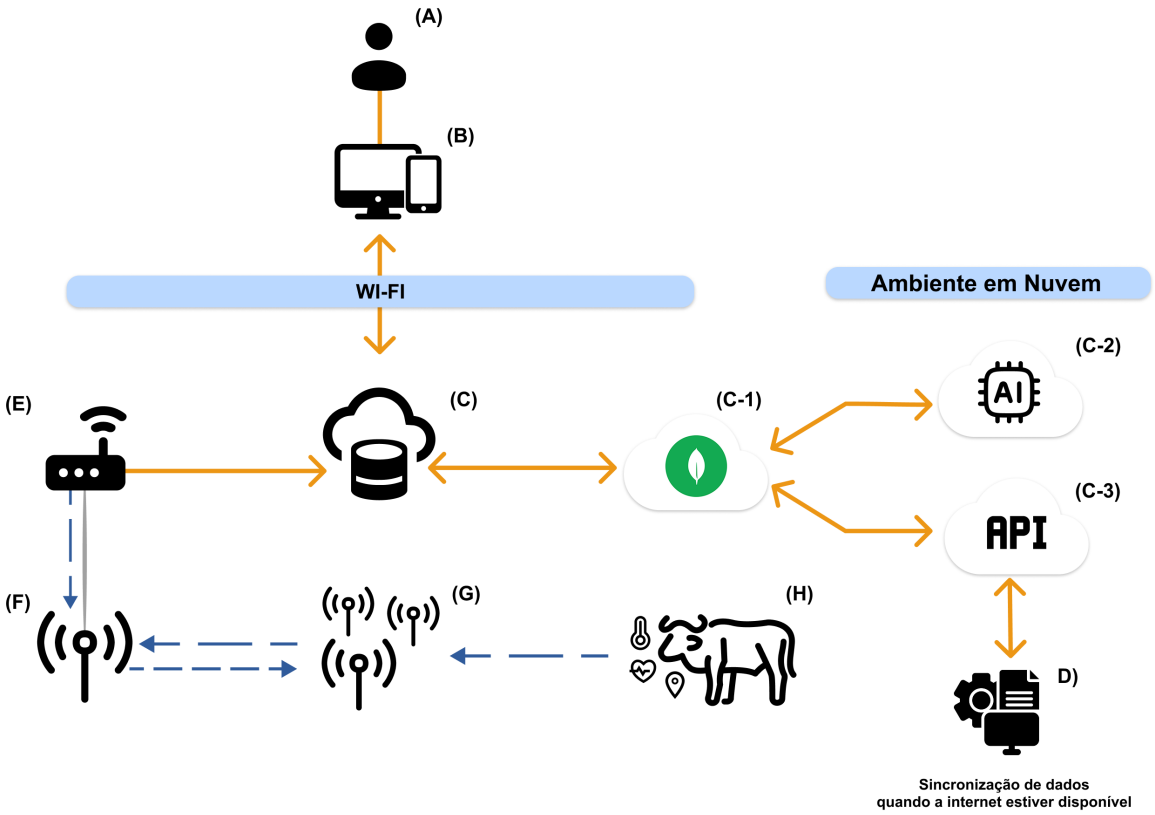
A infraestrutura do Bubatag é organizada em componentes responsáveis pela coleta, transmissão, processamento e visualização dos dados. A Tabela 8 apresenta os principais componentes da arquitetura.

Tabela 8: Componentes de Infraestrutura do Bubatag

Componente	Descrição
Dispositivos e Rede Local	• Microcontrolador ESP32 responsável pela coleta e transmissão dos dados • Sensor DS18B20 para medição da temperatura • Sensor MAX30102 para leitura da frequência cardíaca • Comunicação Wi-Fi para envio das informações à API • Utilização de conexão sem fio para comunicação entre os dispositivos
Infraestrutura em Nuvem	• API REST hospedada na plataforma Render • Banco de dados não relacional MongoDB para armazenamento das informações • Recebimento e processamento das leituras de telemetria • Serviço experimental de Inteligência Artificial para classificação dos dados fisiológicos e geração de relatórios • Comunicação entre os dispositivos e as aplicações por meio de requisições HTTP
Aplicações Web e Mobile	• Interface web desenvolvida com React para visualização e gerenciamento das informações • Aplicação mobile destinada ao acompanhamento dos animais • Visualização de dados de temperatura, frequência cardíaca e localização • Integração das aplicações com a API REST • Utilização de mapas para apresentação dos registros de localização dos animais
Automação e Segurança	• Controle de versão dos repositórios utilizando Git e GitHub • Automação de processos de integração contínua com GitHub Actions • Construção e execução de aplicações em containers utilizando Docker • Publicação da imagem Docker da API no GitHub Container Registry (GHCR) • Comunicação entre os componentes por meio de endpoints da API

8.6 Diagrama de Infraestrutura

Para ilustrar a organização dos componentes e o fluxo de comunicação entre os dispositivos, a API, o banco de dados e as aplicações, apresenta-se o diagrama de infraestrutura do sistema Bubatag.



(A) Usuário
(B) Interface Mobile/Desktop
(C) Banco de dados
(C-1) Banco de dados na nuvem MongoDB
(C-2) Serviço de IA hospedada na nuvem(Render)
(C-3) API hospedada na nuvem(Render)
(D) Ambiente Local(Banco de Dados,IA,API)
(E) Roteador Gigabit AC1300 Archer C6 TP-Link
(F) Antena direcional principal
(G) Antenas direcionais secundárias
(H) Coleira IoT (ESP32, MAX30102, DS18B20 e NEO - 6M)

HTTP
Cabo Coaxial
Sinal 2GHz/5GHz

Figura 11: Diagrama de Infraestrutura do Sistema Bubatag

A arquitetura apresentada é composta pelos seguintes elementos:

- **Coleiras Inteligentes:** Dispositivos responsáveis pela coleta de dados fisiológicos dos animais, utilizando o microcontrolador ESP32 conectado aos sensores DS18B20, para medição da temperatura, e MAX30102, para leitura da frequência cardíaca.
- **Comunicação Wi-Fi:** Utilizada para estabelecer a comunicação entre o ESP32 e a API, permitindo o envio das informações coletadas pelos sensores.

- **API REST:** Responsável por receber as informações de telemetria, processar as requisições e intermediar a comunicação entre os dispositivos, o banco de dados e as aplicações.
- **Banco de Dados MongoDB:** Banco de dados não relacional utilizado para armazenar as informações dos usuários, animais, coleiras, leituras fisiológicas, registros de localização e histórico de estresse.
- **Servidor em Nuvem:** Ambiente responsável por disponibilizar a API e permitir o acesso remoto aos serviços do sistema. Atualmente, a API está hospedada na plataforma Render.
- **Inteligência Artificial (IA):** Serviço experimental responsável pela classificação dos dados fisiológicos dos animais em categorias de estresse e pelo apoio à geração de relatórios. O modelo encontra-se em desenvolvimento e utiliza dados simulados em sua etapa experimental.
- **Aplicação Web:** Interface desenvolvida para permitir o gerenciamento do rebanho, a consulta das informações dos animais e a visualização dos dados fisiológicos e registros de localização em mapas.
- **Aplicação Mobile:** Interface destinada ao acompanhamento dos animais por dispositivos móveis. Sua integração com a API e o banco de dados encontra-se em desenvolvimento.

8.7 Arquitetura de Deploy

A arquitetura de deploy do Bubatag utiliza o GitHub Actions para automatizar etapas de validação, construção e publicação das aplicações. Os processos são definidos separadamente para os repositórios do frontend e da API, conforme apresentado na Figura 12.

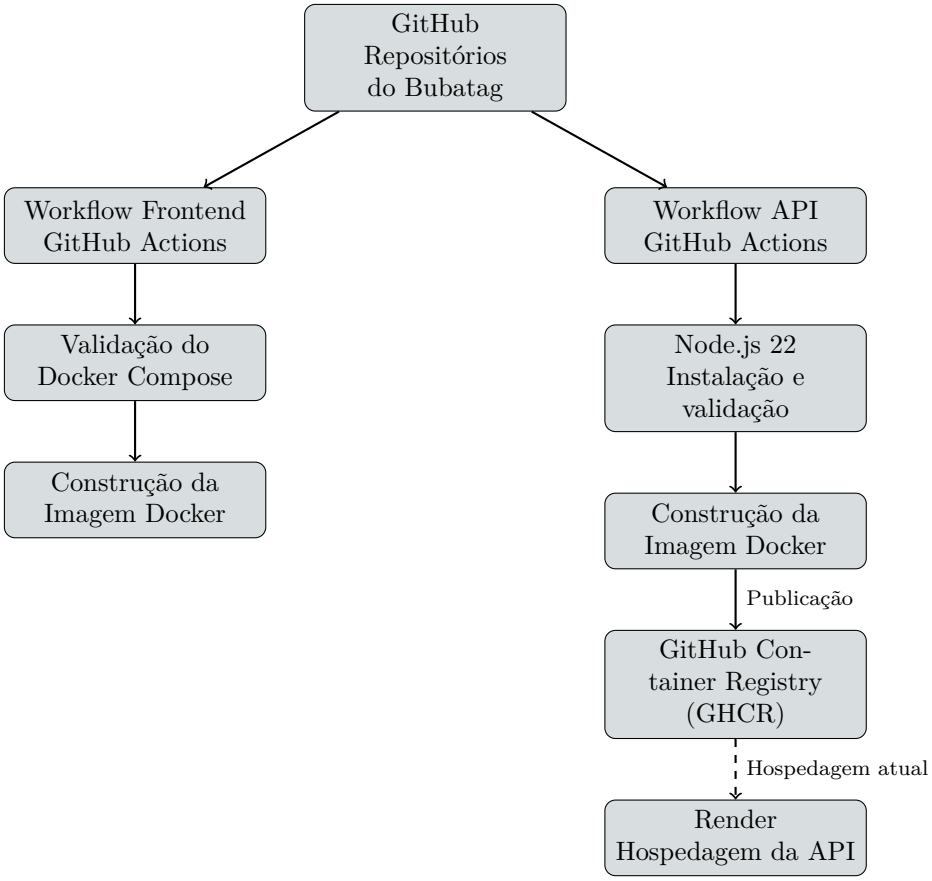


Figura 12: Arquitetura de Deploy do Sistema Bubatag

A arquitetura é composta pelos seguintes elementos:

- **GitHub:** Hospeda os repositórios do projeto, contendo o código-fonte do frontend e da API.
- **GitHub Actions:** Executa os workflows de CI/CD acionados por eventos de *push* e *pull request* na branch principal.
- **Docker:** Responsável pela construção das imagens das aplicações, permitindo seu empacotamento em containers.
- **GitHub Container Registry (GHCR):** Serviço utilizado para armazenar e publicar a imagem Docker da API.
- **Render:** Plataforma de nuvem utilizada para hospedar e executar a API Bubatag.

No frontend, o processo automatizado contempla a validação da configuração do Docker Compose e a construção da imagem Docker. Na API, além da configuração do ambiente Node.js e da instalação das dependências, são realizadas a verificação sintática do código, a construção da imagem e sua publicação no GHCR.

Essa estrutura permite organizar os processos de construção e disponibilização das aplicações, facilitando a manutenção e a evolução da infraestrutura do sistema.

8.8 Métricas e SLA

O monitoramento de desempenho e disponibilidade é importante para avaliar a confiabilidade e a eficiência da infraestrutura do sistema Bubatag. Para isso, são definidos indicadores que poderão ser utilizados em etapas futuras de testes e validação. Como o projeto ainda se encontra em fase de prototipação, não existem medições sistemáticas suficientes para estabelecer níveis de serviço (SLA) ou metas de desempenho comprovadas.

Entre os principais indicadores propostos, destacam-se:

- **Disponibilidade (Uptime):** Percentual de tempo em que a API e as aplicações permanecem acessíveis.
- **Tempo de Resposta:** Tempo necessário para a API processar uma requisição e retornar uma resposta ao cliente.
- **Taxa de Erro:** Percentual de requisições que resultam em falhas durante a comunicação ou o processamento dos dados.
- **Throughput:** Quantidade de requisições que a API consegue processar por unidade de tempo.
- **Frequência de Deploy:** Quantidade de atualizações disponibilizadas em determinado período, considerando os processos de integração e construção automatizados pelo GitHub Actions.
- **Lead Time:** Tempo decorrido entre a implementação de uma alteração no código e sua disponibilização no ambiente de execução.
- **MTTR (Mean Time to Recovery):** Tempo médio necessário para recuperar o funcionamento do sistema após uma falha.

A avaliação desses indicadores permitirá identificar limitações da infraestrutura, acompanhar a evolução do sistema e estabelecer metas de desempenho e disponibilidade compatíveis com as necessidades da bubalinocultura.

9 Conclusão

Este documento reúne os principais artefatos utilizados na documentação e organização do projeto Bubatag, contemplando aspectos estratégicos, visuais, técnicos e operacionais do desenvolvimento de software.

Os artefatos documentados incluem:

- **CANVAS:** Representação do modelo de negócio e da proposta de valor do Bubatag.
- **SWOT:** Análise estratégica das forças, fraquezas, oportunidades e ameaças relacionadas ao projeto.
- **UX/UI:** Documentação das interfaces e da experiência de uso das aplicações web e mobile.
- **Website:** Estrutura, tecnologias e especificações técnicas da plataforma web.
- **Aplicação:** Descrição das funcionalidades, arquitetura e tecnologias utilizadas no sistema.
- **Diagramas UML:** Representação visual da estrutura, dos componentes e das interações do sistema.
- **Diagramas de Banco de Dados:** Representação da organização e dos relacionamentos entre os dados armazenados no MongoDB.
- **DevOps:** Documentação da infraestrutura, containerização, automação de processos e workflows de integração contínua (CI/CD).

A organização desses artefatos proporciona uma visão estruturada do projeto, facilitando a compreensão de sua arquitetura, o acompanhamento do desenvolvimento e a manutenção das aplicações. Além disso, a documentação contribui para a padronização das informações e para a evolução contínua do Bubatag.

Referências

DATE, C. J. *Introdução a Sistemas de Bancos de Dados*. 8. ed. Rio de Janeiro: Elsevier, 2020.

ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. 7. ed. São Paulo: Pearson, 2019.

HEUSER, C. A. *Projeto de Banco de Dados*. 6. ed. Porto Alegre: Bookman, 2009.

KUMAR, S.; CHEN, L. Efficient algorithms for differential equation systems. In: *Proceedings of the International Conference on Computational Science*. Prague, Czech Republic: [s.n.], 2024. p. 112–125.

OLIVEIRA, P.; COSTA, M. Machine learning applications in dynamic systems. *IEEE Transactions on Systems, Man, and Cybernetics*, IEEE, v. 53, n. 8, p. 4567–4580, 2023.

SANTOS, A.; LIMA, B.; FERREIRA, C. Advanced numerical methods for complex system analysis. *Journal of Computational Mathematics*, Springer, v. 45, n. 3, p. 234–251, 2024.

SILVA, J.; ALMEIDA, R.; RODRIGUES, T. Computational modeling of nonlinear phenomena. *Applied Mathematics and Computation*, Elsevier, v. 412, p. 126589, 2025.

WANG, Y.; ZHANG, H.; LEE, K. Comparative study of numerical integration methods. *Numerical Algorithms*, Springer, v. 92, n. 4, p. 1523–1547, 2023.